MediScan

Mini Project Report

Submitted in partial fulfilment of the requirements for the award of the degree in

INTEGRATED M.Sc. PROGRAMME IN COMPUTER SCIENCE DATA SCIENCE

Submitted by

KARTHIK UNNIKRISHNAN

REG. NO: 223142810012

Under the guidance of

Ms. LIJI JOSE P

DEPARTMENT OF COMPUTER SCIENCE - DATA SCIENCE

NIRMALA COLLEGE MUVATTUPUZHA (AUTONOMOUS)

(Affiliated to M.G. University, Kottayam)



NIRMALA COLLEGE MUVATTUPUZHA (AUTONOMOUS)

(Affiliated to M.G. University, Kottayam)

CERTIFICATE

This is to certify that the project entitled "MediScan" submitted in partial fulfilment of the requirements for the award of the degree in Integrated M.Sc. Computer Science – Data Science is a bonafide report of the project done by Karthik Unnikrishnan (Reg.No223142810012) during the academic year 2026-2027.

Internal Guide **Head of the Department**

Examiner

ACKNOWLEDGEMENT

First and foremost, I thank God Almighty for His unwavering grace and blessings, which have been a constant source of strength and inspiration throughout this project.

I would like to express my deep gratitude to our Principal **Rev. Dr. JESTIN K KURIAKOSE**, for giving me the opportunity to learn at this institution. I would like to express my sincere gratitude to my guide, **Ms. LIJI JOSE P**, Assistant professor of department of Integrated MSc Computer Science-Data Science, Nirmala College Muvattupuzha (Autonomous), for the support, guidance and expert insights throughout this project. Her constructive feedback and constant encouragement were instrumental in refining and shaping the project. I truly appreciate the time, effort, and dedication she invested in ensuring the successful completion of this work.

I am also grateful to my institution and faculty members for providing the necessary resources and knowledge that have been instrumental in the development of this project.

A special appreciation goes to my family and friends for their support, motivation and belief in my abilities.

Finally, I would like to acknowledge the online resources, open-source tools, public datasets and research papers that provided valuable insights for the successful implementation of the MediScan system.

Thank you all for your contributions and support.

ABSTRACT

MediScan is an offline medicine identification system. A user uploads a photograph of a medicine pack or of a handwritten prescription, and the system reads the text, finds the medicine in a local database of 503,536 Indian medicines, and shows its salt, strength, manufacturer, price, side effects and interactions. No data leaves the user's computer.

The system uses two TrOCR models fine-tuned by the project. Model A reads printed packs and labels; it was trained on consensus-labelled real crops and synthetic label images, and reads real label crops with 84.9% exact match and 1.83% character error. Model B reads doctors' handwriting; it was trained on 177,057 images from several sources and reads a real-only test set with 14.8% character error and 73.0% exact match. After matching with a brand-word index of 164,550 words, 82.8% of in-database handwritten test words are the first suggestion.

A corrected training loss, leak-free test sets, a precision gate and four honest result levels (Verified, Likely, Not found, Setup error) make the answers trustworthy: on 108 pack photographs the precision of a Verified answer rose from 66.7% to 83.3%. A photograph is processed in about five seconds on an ordinary laptop. The application is a Django website that runs entirely on the local machine.

Keywords: Medicine identification, TrOCR, handwritten prescription recognition, OCR, fuzzy matching, offline application, Django.

CONTENTS

Chapter 1: Introduction **1**
1.1 Need for Study 1
1.1.1 Problem Statement 1
1.2 Objectives 2
1.2.1 Primary Objective 2
1.2.2 Secondary Objectives 2
1.3 Summary 2
Chapter 2: Literature Review **3**
2.1 Introduction 3
2.1.1 Classical and Recurrent OCR 3
2.1.2 Transformer-Based Recognition 3
2.1.3 Text Detection 3
2.1.4 Handwriting Datasets 3
2.1.5 Approximate String Matching 4
2.1.6 Training Techniques 4
2.2 Summary 4
Chapter 3: System Analysis **5**
3.1 Analysis of Dataset 5
3.1.1 Model A: Printed Packs and Labels 5
3.1.2 Model B: Handwritten Prescriptions 5
3.1.3 Medicine Database 6
3.2 Data Preprocessing 6
3.2.1 Image Cleaning 6
3.2.2 Label Cleaning 6
3.2.3 Leak Control 6
3.3 Data Visualization 7
3.4 Analysis of Architecture 8
3.4.1 TrOCR Overview 8
3.4.2 Why Two Models 8
3.5 Project Pipeline 9
3.5.1 Pipeline for Printed Packs (Model A) 11
3.5.2 Pipeline for Handwritten Prescriptions (Model B) 11
3.5.3 Result Levels 12
3.6 Feasibility Analysis 13
3.6.1 Technical Feasibility 13
3.6.2 Economic Feasibility 13
3.6.3 Operational Feasibility 13
3.7 System Environment 13
3.7.1 Software Requirements 13
3.7.2 Hardware Environment 14

Chapter 4: System Design **16**
4.1 Model Building 16
4.1.1 Design Decisions 16
4.1.2 The Training-Loss Problem and Its Fix 16
4.2 Training 17
4.2.1 Common Setup 17
4.2.2 Model A Training 17
4.2.3 Model B Training (Handwritten Prescriptions) 19
4.3 Testing 20
4.3.1 Sanity Check Before Training 20
4.3.2 Testing Method 20
4.3.3 Test Results 21
Chapter 5: Results and Discussion **22**
5.1 Performance Summary: Printed Packs 22
5.1.1 Reading Quality 22
5.1.2 Comparison with the Pre-Trained Reader 22
5.1.3 Effect of the Matching Rules 22
5.2 Performance Summary: Handwritten Prescriptions 23
5.3 Database-Matched Results for Model B 24
5.4 Error Analysis 25
5.5 Speed 25
5.6 Discussion and Limitations 26
Chapter 6: Model Deployment **27**
6.1 Django Web Application 27
6.1.1 Upload Page 27
6.1.2 Result Page 27
6.1.3 Standalone Operation 27
6.2 Git History 28
Chapter 7: Conclusion **29**
Chapter 8: Future Scope and Enhancements **31**
Chapter 9: References **33**
Appendix **34**
Screenshots 34

LIST OF FIGURES

3.1 Size of each source in the Model B training set 7
3.2 Sample images from the Model B training set 7
3.3 Sample synthetic handwriting images 8
3.4 Three RxHandBD test images whose label belongs to a neighbouring image 8
3.5 TrOCR architecture used in both models 9
3.6 End-to-end processing pipeline of MediScan 10
3.7 Processing pipeline for printed packs (Model A) 11
3.8 Processing pipeline for handwritten prescriptions (Model B) 12
4.1 System architecture of MediScan 16
5.1 Model A reading quality on real label crops 22
5.2 Precision and share of photos called Verified 23
5.3 Model B results on the real-only test set 24
5.4 Model B accuracy after database matching 25
6.1 Git commit history of the MediScan repository 28
9.1 Home page 34
9.2 Printed pack result 34
9.3 Handwritten prescription result 35
9.4 Medicine details and interactions 35

CHAPTER 1
INTRODUCTION

MediScan is an offline medicine-identification system. A person takes a photograph of a medicine strip, bottle, box or a doctor's handwritten prescription. The system reads the text in the photograph, finds the medicine in a local database of more than five lakh Indian medicines, and shows its name, salt, strength, manufacturer, price, side effects and known interactions. Every step runs on the user's own computer. No image or text is sent to any outside service.

The project is built around two separate recognition models. Model A reads printed text on medicine packs and labels. Model B reads doctors' handwriting on prescriptions. Both are TrOCR models that were fine-tuned by the project team. They are connected to a Django web application, so the complete system works as a normal website that runs on a local machine.

1.1 NEED FOR STUDY

Medicines are sold in small strips with crowded labels, and many different brands share the same salt. A patient, a caregiver or a pharmacy assistant often needs to answer simple questions quickly: What is this tablet? What is it used for? Can it be taken together with another medicine? Reading a doctor's handwriting adds a second difficulty, because prescriptions are written fast, in many styles, and often use short or unusual spellings.

Online search needs the user to type the name correctly, which is exactly what is hard when the label is small or the handwriting is unclear. Existing mobile applications also depend on an internet connection and on remote servers. This is a problem in places with weak connectivity, and it is a privacy concern because a photograph of a prescription is personal health information.

1.1.1 PROBLEM STATEMENT

To build a fully offline system that (i) reads printed medicine labels and (ii) reads handwritten prescriptions from photographs, (iii) identifies each medicine in a local database even when the reading has small errors, and (iv) tells the user honestly how sure it is, so that a wrong guess is never presented as a confirmed fact.

1.2 OBJECTIVES

1.2.1 PRIMARY OBJECTIVE

To develop MediScan, a Django web application that uses two fine-tuned TrOCR models, one for printed packs and one for handwritten prescriptions, and a local medicine database to identify medicines from photographs without an internet connection.

1.2.2 SECONDARY OBJECTIVES

- Build a clean training set for printed labels using agreement between two readers and synthetic images made from the medicine database.
- Collect and clean a large handwriting set (177,057 images) that includes real doctors' handwriting, and keep a test set that contains only real prescription words.
- Find and fix the training-loss problem in the TrOCR library so that both models learn correctly.
- Add a matching layer (brand-word index with fuzzy matching) that turns a noisy reading into a database medicine.
- Show results in four honest levels: Verified, Likely, Not found and Setup error.
- Keep a photo-to-answer time of about five seconds on an ordinary laptop.

1.3 SUMMARY

This chapter introduced MediScan, explained why an offline medicine reader is useful and stated the problem and objectives. Chapter 2 reviews related work. Chapter 3 analyses the data, the architecture and the pipeline. Chapter 4 describes how the models were built, trained and tested. Chapters 5 and 6 give the results and the deployment. Chapters 7 and 8 give the conclusion and the future scope.

CHAPTER 2
LITERATURE REVIEW

2.1 INTRODUCTION

This chapter reviews the work that MediScan builds on: optical character recognition (OCR) for printed and handwritten text, transformer-based recognisers, text detection, and approximate string matching. Numbers in square brackets refer to the list in Chapter 10.

2.1.1 CLASSICAL AND RECURRENT OCR

Early OCR engines such as Tesseract [1] split a page into characters or short pieces and classify each one. They work well on clean printed pages but fail on stylised fonts, curved packaging and foil, which are common on medicine strips. Recurrent models changed this by reading a whole text line. The CRNN model [2] combines a convolutional feature extractor with a recurrent network, and Connectionist Temporal Classification (CTC) training [3] removes the need to mark where each character starts and ends. These models are fast but have a weak language model, so they often return a plausible-looking but wrong spelling for rare words such as brand names.

2.1.2 TRANSFORMER-BASED RECOGNITION

The Vision Transformer [4] cuts an image into fixed patches and processes them with self-attention [5]. TrOCR [6] uses this idea for text reading. A Vision Transformer encoder looks at the image, and a text-decoder transformer writes the characters one piece at a time. Both parts are pre-trained, so the model needs relatively little task data to adapt. Public checkpoints exist for printed text and for handwritten text. This is why MediScan uses the printed checkpoint for packs (Model A) and the handwritten checkpoint for prescriptions (Model B).

2.1.3 TEXT DETECTION

TrOCR reads one cropped word or line at a time, so a separate step must first find where the text is. CRAFT [7], the detector used in the EasyOCR library, finds text regions by predicting the area of each character and the links between characters. MediScan uses only this detection part and lets TrOCR do the reading.

2.1.4 HANDWRITING DATASETS

The IAM Handwriting Database [8] is the standard English handwriting benchmark, with lines written by many writers. It is useful for learning general handwriting but contains ordinary sentences, not medicine names. Prescription-specific sets are small, so MediScan combines several

sources: real doctors' handwriting (RxHandBD and a Bangladeshi prescription set), a large word-image set (SSL-HWD), IAM lines, pen-written medicine names and synthetic handwriting rendered from the project's own medicine database.

2.1.5 APPROXIMATE STRING MATCHING

A reading is rarely perfect, so the result must be matched against a list of known names. The edit distance [9] counts the smallest number of insertions, deletions and substitutions between two strings. MediScan uses the RapidFuzz library [10] to compute similarity scores on a 0–100 scale against a brand-word index. Section 3.5 explains the extra rules that are placed on top of this score to avoid wrong matches.

2.1.6 TRAINING TECHNIQUES

Fine-tuning uses the AdamW optimiser [11] and mixed-precision (fp16) arithmetic [12] to reduce memory and time. The Hugging Face Transformers library [13] provides the TrOCR code. The project found that the built-in loss of this library shifts the labels twice for the checkpoints used here, which hurts learning; the correction is described in Section 4.1.

2.2 SUMMARY

Classical OCR is weak on stylised labels, recurrent models are weak on rare words, and TrOCR combines a strong image encoder with a language-aware decoder. TrOCR needs a detector and a matching layer in front of and behind it to work as a complete prescription and label reader, and MediScan supplies both. The works reviewed here concentrate on recognition accuracy. MediScan adds an offline database, a matching layer and honest confidence levels around the recognisers, and covers printed packs and handwritten prescriptions in one application.

CHAPTER 3
SYSTEM ANALYSIS

3.1 ANALYSIS OF DATASET

Two separate training sets were prepared, one for each model. Model A needs images of printed text from medicine packs. Model B needs images of handwritten words and lines, with as much real doctors' handwriting as could be found.

3.1.1 MODEL A: PRINTED PACKS AND LABELS

Real pack photographs have no typed answer, so the labels were produced in two ways. First, two stock (pre-trained) readers read every crop, and a crop was kept for training only when both readers agreed on the text. This is called consensus pseudo-labelling. Second, synthetic label images were rendered from the names, salts and strengths stored in the medicine database, using many fonts, backgrounds and distortions. The final split between training, validation and test images was made so that no pack appeared in more than one part (a leak-free split).

3.1.2 MODEL B: HANDWRITTEN PRESCRIPTIONS

Model B needed far more data because handwriting varies greatly between writers. The final set has 177,057 images from several sources, listed in Table 3.1. The two sources of real doctors' handwriting (RxHandBD and the Bangladeshi prescription set) are the closest to the real task. All test images come only from these real sources, so the test result is not inflated by synthetic or printed-style images. Each test row also carries a flag, seen_in_train, which says whether the same word occurs in the training text. This allows a separate score for words the model has never seen.

Table 3.1: Main sources of the Model B training set

Source	**Images**	**What it contains**
SSL-HWD words	94,705	Large collection of handwritten word images
Synthetic handwriting	60,000	Medicine names from the database drawn with 30 handwriting fonts
IAM lines	9,397	English handwritten text lines
RxHandBD	4,463	Real doctors' handwriting, medicine names
BD prescription	2,903	Real doctors' handwriting, prescription words
Pen-written medicine names	1,696	Medicine names written with a pen
Other small sources	—	Medical word lists, illegible-prescription samples and others
Total	**177,057**	After removing duplicates (MD5 hash)

3.1.3 MEDICINE DATABASE

The knowledge base is two SQLite files. The file medicines.sqlite has a table named medicines with 503,536 rows and the columns id, name, salt, manufacturer, price and source. The file drugs.sqlite holds side effects and drug interactions. The table below shows the layout of the medicines table. The names in medicines.sqlite are also the source of the brand-word index described in Section 3.5, which has 164,550 unique brand words for Model A.

Table 3.2: Layout of the medicines table

Column	Type	Meaning
id	INTEGER	Primary key
name	TEXT	Brand name with strength and form
salt	TEXT	Active ingredient(s) with strength
manufacturer	TEXT	Company
price	REAL	Price in rupees
source	TEXT	Dataset the row came from

3.2 DATA PREPROCESSING

3.2.1 IMAGE CLEANING

- Photo orientation is fixed using the EXIF tag so that phone photographs are not rotated.
- Handwriting images from RxHandBD and the pen-written set are auto-cropped to the ink and resized to at most 1024 × 384 pixels.
- Exact duplicate images are removed using an MD5 hash.
- Every image is finally resized to 384 × 384 pixels by the TrOCR processor.

3.2.2 LABEL CLEANING

Labels keep their original upper and lower case during training, but the character error rate and the exact-match score are computed without case, because for medicine names “OMEP” and “Omep” are the same word. A stop list removes header words such as “Rx”, “Date” and “Name” from the results at inference time.

3.2.3 LEAK CONTROL

The Model B test set was checked so that its source is only RxHandBD or the Bangladeshi prescription set. A script (rx_shift_check.py) also compared each prediction with its own label and

with the label of the previous and next image. This showed that a small number of RxHandBD labels in the test set belong to the neighbouring image (Section 5.3).

3.3 DATA VISUALIZATION

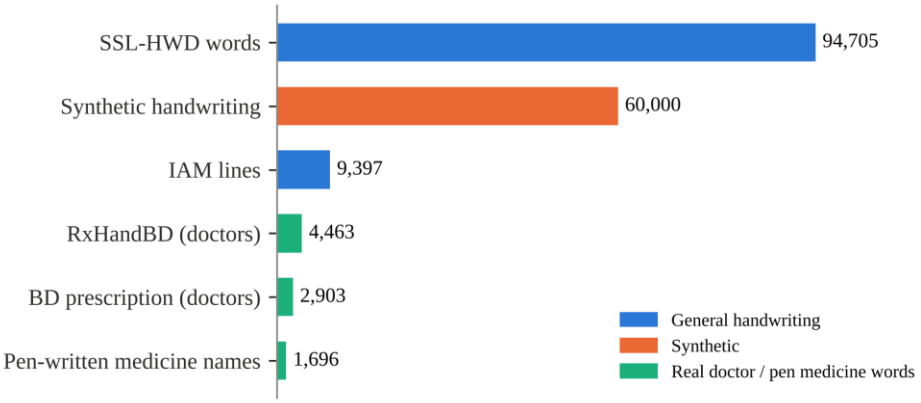


Fig 3.1: Size of each source in the Model B training set



Fig 3.2: Sample images from the Model B training set



Fig 3.3: Sample synthetic handwriting images



Image rxhandbd_004509
Label in dataset: Doxycap
Word actually shown: Omep

Image rxhandbd_004510
Label in dataset: Robac
Word actually shown: Doxycap

Image rxhandbd_004511
Label in dataset: Omep
Word actually shown: Rolac

Fig 3.4: Three RxHandBD test images whose label belongs to a neighbouring image

3.4 ANALYSIS OF ARCHITECTURE

3.4.1 TROCR OVERVIEW

Both models use the TrOCR design (Fig 3.5). The image is cut into 16 × 16 patches, giving 576 patches for a 384 × 384 input. A Vision Transformer encoder turns the patches into a sequence of feature vectors. A text decoder then writes the answer one token at a time, looking at the encoder output through cross-attention. The decoder finishes by writing an end token.

3.4.2 WHY TWO MODELS

Printed text and handwriting are very different. A model trained on one tends to lose accuracy on the other. Keeping them apart also lets each model be trained, tested and replaced on its own, and lets the application choose the model that matches the type of photograph.

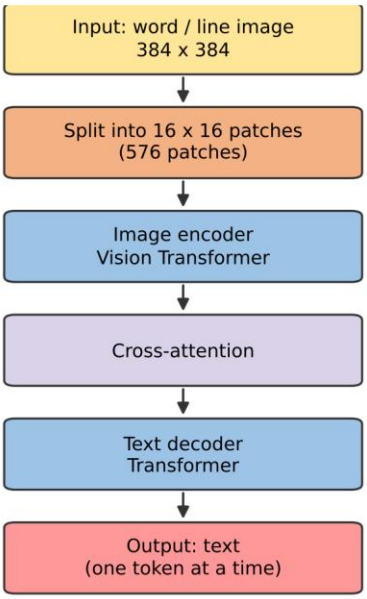


Fig 3.5: TrOCR architecture used in both models

Table 3.3: The two models

Item	**Model A**	**Model B**
Task	Printed packs and labels	Handwritten prescriptions
Starting checkpoint	TrOCR-large-printed	TrOCR-base-handwritten
Training data	Consensus-labelled crops plus synthetic labels	177,057 images, real doctors' handwriting included
Stored size	About 2.44 GB	Smaller (base model)
Detector	EasyOCR (detection only)	EasyOCR (detection only)
Matching index	164,550 brand words	Same index

3.5 PROJECT PIPELINE

This section describes the complete processing from photograph to answer. The two pipelines share the same matching index and the same database. The processing of MediScan has five stages: image preparation, text detection, reading and parsing, database matching with safety gates, and result display. Fig 3.6 shows how a photograph moves through these stages, from the upload to the final result page.

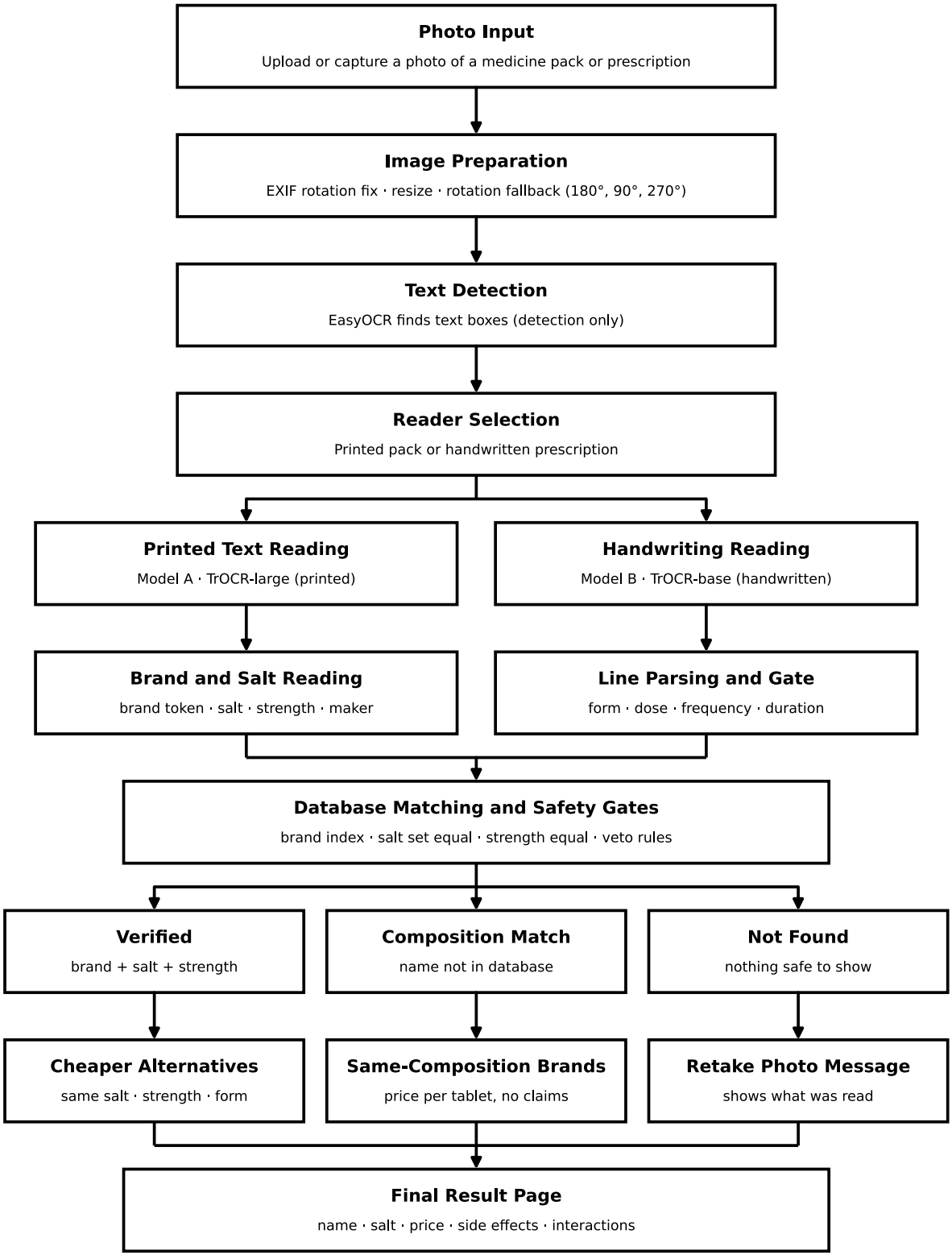


Fig 3.6: End-to-end processing pipeline of MediScan

Fig 3.7 and Fig 3.8 show the two pipelines. They share the same matching index and the same database, and nothing leaves the computer.

3.5.1 PIPELINE FOR PRINTED PACKS (MODEL A)

1. **Fix the photo.** The EXIF tag is read and the photo is rotated to upright.
2. **Find the text.** EasyOCR detection finds the text boxes. Only the boxes are used; EasyOCR does not read the text.
3. **Read each box.** Model A reads every cropped box.
4. **Match.** Each reading is matched against the brand-word index with RapidFuzz. A reading that is exactly a brand word is accepted as an exact brand.
5. **Corroborate.** The salt and strength printed on the pack are compared with the database entry of the matched brand. A scoped compound guard stops a long word from matching a short brand inside it.
6. **Rotate if needed.** If nothing is found, the photo is tried at 180°, then at 90° and 270° within a time budget of 15 seconds.
7. **Decide the level.** The result is Verified, Likely, Not found or Setup error (see Result Levels below).

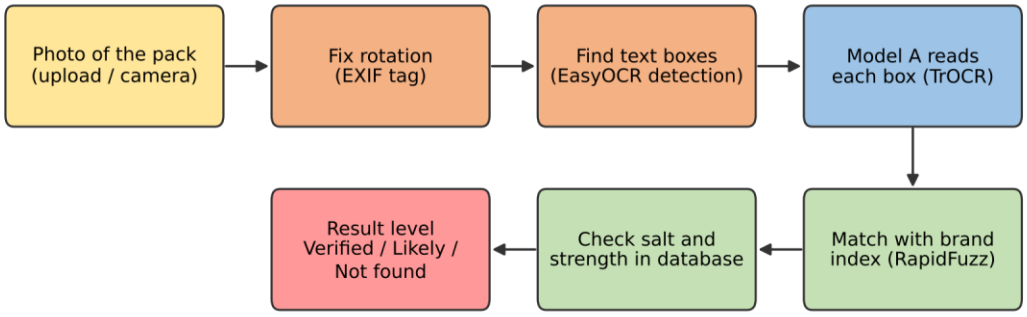


Fig 3.7: Processing pipeline for printed packs (Model A)

3.5.2 PIPELINE FOR HANDWRITTEN PRESCRIPTIONS (MODEL B)

1. **Find the words.** EasyOCR detection returns boxes of at least 12 × 12 pixels, with a maximum of 40 boxes per photo. The boxes are grouped into lines.
2. **Read each box.** Model B reads the boxes in batches of 16 with beam search (2 beams) and fp16.

3. **Parse each line.** parse_line extracts the form (tablet, capsule, syrup and so on), the dose, the frequency (for example 1-0-1) and the duration (for example 7 days). Header words are removed with the stop list.
4. **Match the medicine word.** The reading is matched with the brand-word index.
5. **Precision gate.** A word is shown as a medicine only if it passes the gate in Table 3.4.
6. **Show the rest.** Text that did not become a medicine is shown separately as "other text read".

Table 3.4: Precision gate for handwritten medicine names

Case	Condition
A	Word has at least 4 letters, the match score is at least 85, and a supporting item (dose, form or frequency) is on the same line
B	Match score is at least 95, the word has at least 5 letters and it is not on the stop list

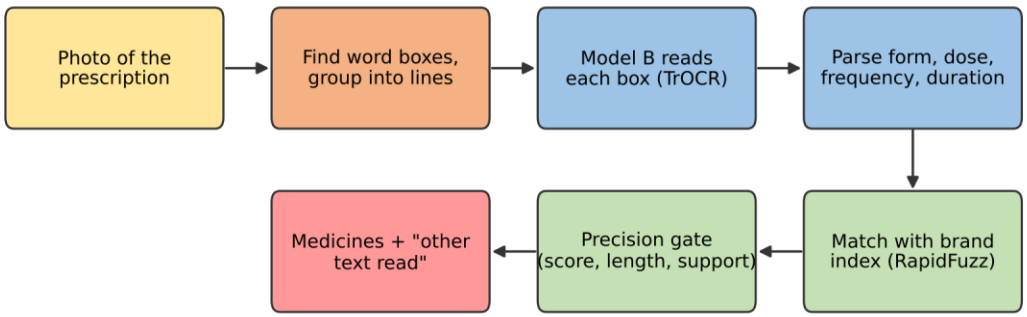


Fig 3.8: Processing pipeline for handwritten prescriptions (Model B)

3.5.3 RESULT LEVELS

A lookup that returns the wrong medicine is worse than no answer. For this reason the application shows one of four states. Verified means the brand is an exact match and the salt printed on the pack agrees with the database. Likely means there is a good match but not enough supporting evidence. Not found means no safe match exists. Setup error means a model or database file is missing or unreadable.

3.6 FEASIBILITY ANALYSIS

3.6.1 TECHNICAL FEASIBILITY

All components are open source and run on Python. The trained models are loaded once at start-up. Photographs are processed on the CPU in about five seconds on average, and a GPU is needed only for training. The project was completed with the free GPU time provided by Kaggle for training and a normal laptop for the application.

3.6.2 ECONOMIC FEASIBILITY

No paid service is needed. The software is free and the data sets are public or generated by the project. The only cost is the computer, which the user already has. Because the system runs offline, there is no recurring cost.

3.6.3 OPERATIONAL FEASIBILITY

The user opens a web page, uploads a photograph and reads the result. No training is needed to use it. The wording of the four result levels tells the user how much to trust each answer, and the interface advises checking with a pharmacist or doctor.

3.7 SYSTEM ENVIRONMENT

3.7.1 SOFTWARE REQUIREMENTS

The system environment specifies both software and hardware specifications.

- Tool: VS Code, Antigravity IDE, Jupyter Notebook (Kaggle notebooks for training)
- Python: Version 3.9.13
- Operating System: Windows 11
- Front End: HTML5, CSS3 (Django templates)
- Back End: Python with Django
- Database: SQLite (medicines.sqlite, drugs.sqlite)

The entire project was developed using Python, a high-level programming language known for its simplicity and versatility. Various software used for the development of this application are the following:

- **PyTorch:** The primary deep learning framework. It handled tensor computations, model loading, fp16 training on the GPU and CPU inference for the two fine-tuned TrOCR models (Model A and Model B).

- **Hugging Face Transformers**: Provided the TrOCR code (Vision Transformer encoder and text decoder), the tokenizer and the image processor. It was run in offline mode so that no internet is needed after setup.
- **EasyOCR**: Used only for text detection. It finds the boxes around words and lines on the pack or prescription, and each box is then read by TrOCR.
- **RapidFuzz**: Provided fast fuzzy string matching. It matched the text read by the models with the brand-word index of 164,550 words and gave the similarity score from 0 to 100.
- **SQLite**: Stored the medicine knowledge base. The file medicines.sqlite holds 503,536 medicines (name, salt, manufacturer, price), and drugs.sqlite holds side effects and drug interactions.
- **Django**: Served as the web application framework. It manages the upload page, loads the models once at start-up, calls the recognisers, builds the result with its four levels and renders the result page.
- **Pillow**: Used for image handling. It opened the photographs, fixed their rotation using the EXIF tag, cropped and resized the images.
- **NumPy and pandas**: Handled array processing and the data tables (train, validation and test files) during data preparation and evaluation.
- **Matplotlib and Graphviz**: Used for the result charts and the diagrams in this report.
- **Kaggle Notebooks:** Provided the GPU for fine-tuning both models, with checkpoint and resume support.
- **GitHub:** Used for version control and source code management. It tracked code changes and managed the project history.

3.7.2 HARDWARE ENVIRONMENT

The system's hardware specifications are defined across three tiers to ensure that the application can run on an ordinary computer without internet, and that the models can be trained on a free GPU.

Minimum Configuration (CPU-only operation):

- Processor: Quad-core 2.0 GHz or faster (Intel Core i3 / AMD Ryzen 3 or equivalent)
- RAM: 8 GB
- Storage: 10 GB free space (for the models of about 3 GB, the databases and the libraries)

- GPU: Not required (CPU-based inference is supported for both models)
- Camera: Any phone or camera that gives a clear photo of the pack or prescription
- Illumination / Environment: Good ambient light without strong glare on foil or glossy packs
- Network: Localhost connection for serving the web application; no internet needed

Recommended Configuration (for faster processing):

- Processor: Six-core 3.0 GHz or faster (Intel Core i5 / AMD Ryzen 5 or better)
- RAM: 16 GB or higher
- Storage: 20 GB+ free SSD space
- GPU: NVIDIA GPU with 4 GB+ VRAM and CUDA support (speeds up reading of the boxes)
- Camera: 12 MP or better phone camera
- Illumination / Environment: Uniform, diffuse lighting to avoid shadows and reflections
- Network: Local network, if the phone uploads the photo to the laptop

Configuration Used for Development & Deployment:

- Processor: 12th Gen Intel(R) Core (TM) i5-12450HX
- RAM: 20 GB
- Storage: 474.7 GB SSD
- GPU: NVIDIA GeForce RTX 2050, 4GB VRAM
- Operating System: Windows 11
- Training platform: Kaggle GPU notebook, fp16, about 10.5 hours per training run
- Network: Not needed to run the application; internet was used only to download datasets and to train on Kaggle

CHAPTER 4
SYSTEM DESIGN

4.1 MODEL BUILDING

4.1.1 DESIGN DECISIONS

- **Two models, one application.** Each recogniser is trained and tested alone and then connected through a small Python class (PrintedRecogniser, HandwrittenRecogniser).
- **Detector and reader are separate.** EasyOCR only finds boxes. TrOCR reads them. This keeps the reading quality in the fine-tuned models.
- **Matching after reading.** The model gives a spelling. The database decides which medicine it is. This lets a small reading error still end in the right medicine.
- **Honest wording.** The result level is decided by fixed rules, not by the model's confidence alone.
- **Everything local.** Models, index and databases are files on the same machine.

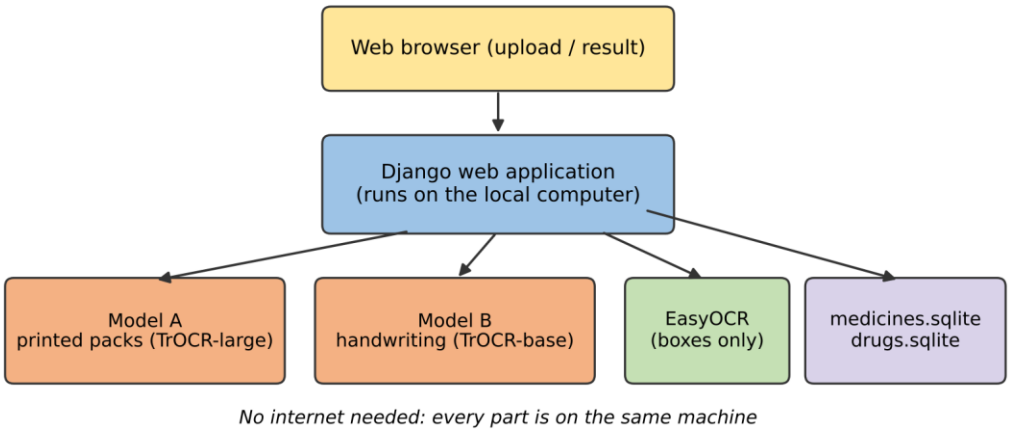


Fig 4.1: System architecture of MediScan

4.1.2 THE TRAINING-LOSS PROBLEM AND ITS FIX

Before long training, a sanity check compared the loss that the library computes with a hand-written cross-entropy loss on the same batch. The two values were very different: for Model B the library gave 18.94 and the correct loss was 3.30, and for Model A the library gave 16.1 and the correct loss was 0.239. The reason is that the library shifts the labels one more step inside its loss, while the batches already carry shifted labels. A model trained with this loss learns to predict the wrong position.

The fix applied to both models was:

- compute the cross-entropy loss directly on correctly shifted labels;
- use the separator token as the start token (START = tok.sep_token_id) and remove the leading start symbol from the labels;
- pass decoder_start_token_id, eos_token_id and pad_token_id explicitly to generate().

After this change the loss started at a sensible value and the character error rate improved from the first epoch. This was important enough to test before every training run.

4.2 TRAINING

4.2.1 COMMON SETUP

Table 4.1: Training settings common to both models

Setting	Value
Input size	384 × 384 pixels (576 patches of 16 × 16)
Precision	fp16 mixed precision
Memory saving	Gradient checkpointing
Optimiser	AdamW
Loss	Own cross-entropy on shifted labels
Resume support	latest.pt every 2,000 steps; best/ for the best validation model
Platform	Kaggle “Save & Run All (Commit)”, so that files are kept after the run

4.2.2 MODEL A TRAINING

Model A reads printed text on medicine packs. It starts from the stock microsoft/trocr-large-printed checkpoint (about 558 million parameters, 24-layer Vision Transformer encoder and 12-layer text decoder) and is fine-tuned on label crops. Training was done on a Kaggle notebook with a Tesla T4 GPU (15 GB).

4.2.2.1 TRAINING DATA

- Training set: 59,529 images (2,551 real crops and 56,978 synthetic label images)
- Validation set: 3,355 images (333 real crops, the rest synthetic)
- Test set: 331 real crops only (PharmAI tablet names: 247, Kaggle drug-name crops: 84)
- Labels are converted to UPPERCASE, because the database matching ignores case and this makes learning easier

- Real crops are augmented during training (colour jitter, small rotation, shift, scale and shear, blur). Synthetic images are used as they are

4.2.2.2 TRAINING CONFIGURATION

- Starting model: microsoft/trocr-large-printed
- Input size: 384 × 384 pixels (576 patches of 16 × 16)
- Maximum label length: 40 tokens
- Batch size: 4 images per step with 8 accumulation steps (effective batch size 32)
- Optimiser: AdamW with weight decay 0.01
- Learning-rate schedule: warm-up for the first 5% of the steps, then straight-line decay down to 5% of the peak value
- Gradient clipping: 1.0
- Precision: fp16 mixed precision with a gradient scaler
- Memory saving: gradient checkpointing
- Loss: our own cross-entropy loss on correctly shifted labels. The sanity check showed the library loss as 16.100 and our loss as 0.239 on the same batch, because the library shifts the labels a second time
- Start token: the separator token (id 2), as used by the stock model, with explicit end and padding tokens
- Time limit: the notebook saves and stops itself after 10.5 hours (4.5 hours on Colab), and it can be run again to continue
- Save points: a resume file (latest.pt) every 4,000 batches and a best-model folder

4.2.2.3 TWO-PHASE TRAINING

- Phase 1 (learn from everything): 1 epoch, peak learning rate 4e-5. Each real crop is repeated 3 times and all synthetic images are added (about 64,600 images per epoch)
- Phase 2 (focus on real crops): 3 epochs, peak learning rate 1.5e-5. Each real crop is repeated 4 times and 10,000 freshly chosen synthetic images are added each epoch (about 20,200 images per epoch)

- At the end of every epoch the model is checked on the 333 real validation crops and on 500 synthetic validation images. The model with the lowest real-validation character error rate is kept as the best model

4.2.3 MODEL B TRAINING (HANDWRITTEN PRESCRIPTIONS)

Model B reads doctors' handwriting. It starts from the stock microsoft/trocr-base-handwritten checkpoint and is trained with the same code as Model A, with changes that suit handwriting.

4.2.3.1 TRAINING DATA

- Total: 177,057 images from several sources (Table 3.1)
- Training set: 173,164 images
- Validation set: 2,031 images
- Test set: 1,862 real doctors' handwriting images only (BD prescription words 747 and RxHandBD 1,115)
- Labels keep their original upper and lower case. The character error rate and exact match are computed without case
- The best model is chosen on the validation set of real doctors' words (val_doc), because that is the task the application needs

4.2.3.2 TRAINING CONFIGURATION

- Starting model: microsoft/trocr-base-handwritten
- Input size: 384 × 384 pixels
- Maximum label length: 48 tokens
- Batch size: 8 images per step with 4 accumulation steps (effective batch size 32)
- Optimiser: AdamW, fp16 mixed precision, gradient checkpointing, same loss correction as Model A
- Loss check before training: the library loss was 18.94 and our loss was 3.30
- Learning rates and number of epochs per phase: {{fill from the Model B training notebook}}
- Time limit: 10.5 hours per run
- Save points: resume file every 2,000 steps and a best-model folder

4.2.3.3 PHASED TRAINING

- Early phase: a mix of the large word-image sources (SSL-HWD, IAM, synthetic names) to learn general handwriting
- Later phases: more weight on the real doctors' handwriting (RxHandBD and BD prescription) and the synthetic medicine names
- Training log: in the last phase (epoch 4), the validation set of real doctors' words gave a character error rate of 2.60% and an exact match of 92.1%

4.3 TESTING

4.3.1 SANITY CHECK BEFORE TRAINING

Before every long run, a one-minute check is done on 48 real validation crops. It compares the library loss with our loss and measures the untrained stock model.

- Model A: library loss 16.100, our loss 0.239. The untrained stock model already reads the easy validation crops well (character error 0.9%, exact match 91.7%), so the stock reader is a strong starting point
- Model B: library loss 18.94, our loss 3.30. The untrained handwriting model gave a character error of 16.9% and an exact match of 29.2% on the check sample

4.3.2 TESTING METHOD

- Both models are tested only after training has finished, on the held-out test set that was never used for training or for choosing the best model.
- The best saved model is loaded, and the test crops are read with beam search (4 beams).
- Two metrics are reported: character error rate (edit distance divided by the length of the true text) and exact match (percentage of readings that are exactly right, ignoring case).
- The results are given for each data source and, for Model B, also for words seen and never seen in the training text (the seen_in_train flag).
- Every wrong prediction is written to errors_test.tsv, which is opened and studied to find the common mistakes.
- Model B is also tested after database matching (db_eval_B.py), and a separate script (rx_shift_check.py) checks the RxHandBD labels for a shift to the neighbouring image.

4.3.3 TEST RESULTS

- Model A (331 real crops): character error 1.83%, exact match 84.9%. The Kaggle crops (84) gave 0.81% error and 92.9% exact match. The PharmAI crops (247) gave 2.31% error and 82.2% exact match
- Model B (1,862 real images): character error 14.8%, exact match 73.0%. BD prescription words (747) gave 3.2% error and 95.7% exact match. RxHandBD (1,115) gave 22.3% error and 57.8% exact match. Words seen in training gave 82.8% exact match and never-seen words gave 38.9%.

CHAPTER 5
RESULTS AND DISCUSSION

5.1 PERFORMANCE SUMMARY: PRINTED PACKS

5.1.1 READING QUALITY

On 331 real label crops Model A reads 84.9% exactly, with a character error rate of 1.83%. The PharmAI crops give 82.2% exact and 2.31% CER; the Kaggle crops give 92.9% exact and 0.81% CER (Fig 5.1).

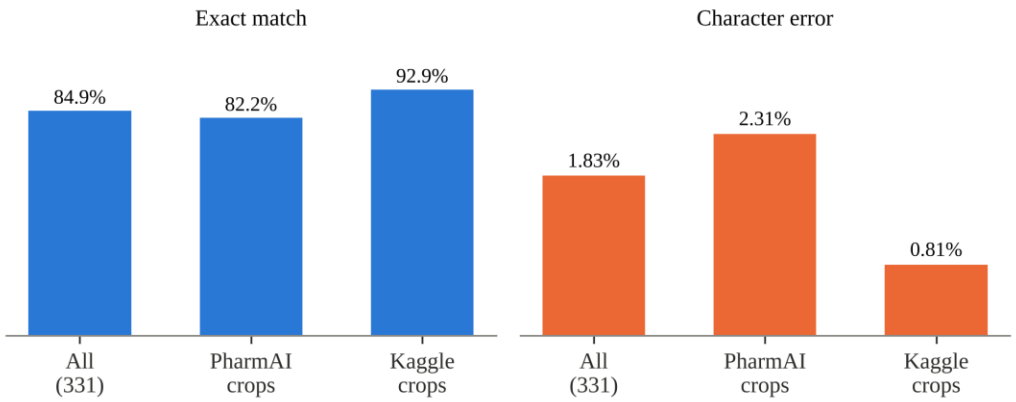


Fig 5.1: Model A reading quality on real label crops

5.1.2 COMPARISON WITH THE PRE-TRAINED READER

On the comparison photo set, the pre-trained printed reader found the correct medicine in its top 1, top 3 and top 5 suggestions in 41.3%, 54.8% and 60.3% of photos. The fine-tuned Model A reached 50.0%, 65.9% and 66.7% (Table 5.1). Fine-tuning therefore improves the final medicine lookup, not only the text reading.

Table 5.1: Pre-trained reader compared with fine-tuned Model A (correct medicine within top k)

Reader	Top 1	Top 3	Top 5
Pre-trained printed reader	41.3%	54.8%	60.3%
Fine-tuned Model A	50.0%	65.9%	66.7%

5.1.3 EFFECT OF THE MATCHING RULES

To check the matching rules without fooling ourselves, the 108 benchmark photos were divided in two halves. The rules were tuned on one half and then checked on the other, untouched half. With

the final matcher, top-1, top-3 and top-5 accuracy on all 108 photos rose from 66.7%, 70.4% and 71.3% with plain word matching to 68.5%, 75.0% and 77.8% (Table 5.2).

Table 5.2: Plain word matching compared with the final matcher (all 108 photos)

Matcher	Top 1	Top 3	Top 5
Plain word matching	66.7%	70.4%	71.3%
Final matcher	68.5%	75.0%	77.8%

The more important gain is in how reliable the word “Verified” is. With plain matching every photo was called verified, but only 63.0% of those answers were right on the untouched half. With the final matcher, 61.1% of untouched photos are called Verified and 78.8% of them are correct. On all photos the precision of Verified rises from 66.7% to 83.3% (Fig 5.2). The system now says “Likely” or “Not found” for photos it is unsure about, instead of giving a confident wrong answer.

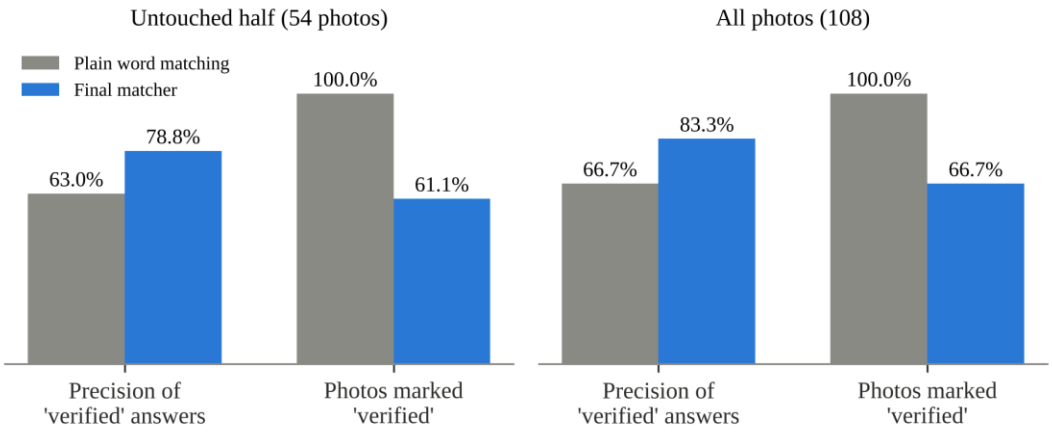


Fig 5.2: Precision and share of photos called Verified

5.2 PERFORMANCE SUMMARY: HANDWRITTEN PRESCRIPTIONS

On the real-only test set of 1,862 images, Model B has a character error rate of 14.8% and reads 73.0% of images exactly (Fig 5.3, Table 5.3). Printed-style Bangladeshi prescription words are read very well (95.7% exact, 3.2% CER). The RxHandBD doctors’ handwriting is harder (57.8% exact, 22.3% CER). Words that appear in the training text are read much better (82.8% exact) than words that never appeared (38.9% exact). This gap is expected for a model that has learnt a vocabulary, and the vocabulary of the application is the medicine database, which is why the matching layer matters.

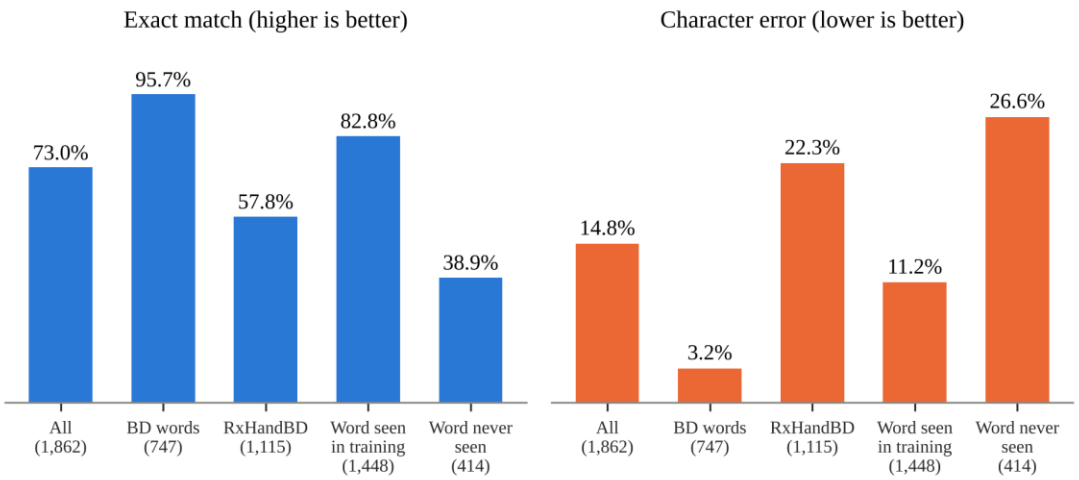


Fig 5.3: Model B results on the real-only test set

Table 5.3: Model B test results

Group	Images	Exact match	CER
All real test images	1,862	73.0%	14.8%
BD prescription words	747	95.7%	3.2%
RxHandBD (doctors)	1,115	57.8%	22.3%
Word seen in training	1,448	82.8%	11.2%
Word never seen	414	38.9%	26.6%

5.3 DATABASE-MATCHED RESULTS FOR MODEL B

The test words that exist in the medicine database (702 words) were passed through the matcher. The model's own reading is exactly right for 80.5% of them. After database matching, the right name is the first suggestion for 82.8%, within the first three for 84.2% and within the first five for 84.6% (Fig 5.4). Matching therefore recovers a small number of near-miss readings, and the top-3 list is almost as good as the top-5 list.

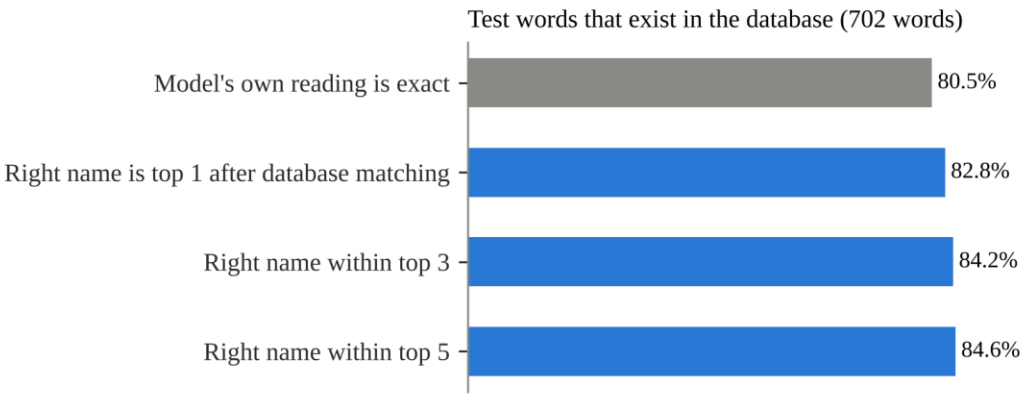


Fig 5.4: Model B accuracy after database matching

5.4 ERROR ANALYSIS

Many remaining errors come from three causes: very short or cursive words, brand names that differ by one letter from another brand, and test labels that belong to a neighbouring image. The last cause was checked with rx_shift_check.py on the RxHandBD test images, sorted by image number. Out of 1,115 images, the reading matches the image's own label in 760 cases, the label of the previous image in 28 cases and the label of the next image in 1 case; in 326 cases it matches none of the three. The shifted labels are mostly among the first 100 images. The effect on the overall score is small, so the model was not retrained, but the exact-match figure for RxHandBD is slightly lower than the true value (Fig 3.4 shows examples).

For the whole application, the first runs of Model B showed English words and header text shown as medicines. This was corrected with the precision gate (Table 3.4) and the stop list. Text that does not pass the gate is displayed under "other text read", so the user still sees everything the model read.

5.5 SPEED

On 12 real prescription photographs the first version of the application took 14.9 seconds on average and 45.2 seconds for the slowest photo. After limiting the number of boxes, using batches of 16, fp16 reading and 2 beams, the average fell to 5.1 seconds and the slowest to 15.4 seconds (Table 5.4).

Table 5.4: Time per photograph, Model B (12 prescription photos)

Version	Average	Slowest
First version	14.9 s	45.2 s
After speed-up	5.1 s	15.4 s

5.6 DISCUSSION AND LIMITATIONS

- Whole-prescription accuracy is not yet measured. The figures above are for single words and for lookups. A twelve-photo ground-truth file for full prescriptions (precision and recall of the medicines listed).
- Handwriting in the wild is hard. Words never seen in training are read correctly less than half the time. The matching layer helps only when the name exists in the database.
- Interactions between prescription medicines have not been confirmed on real prescriptions and should be treated as a future check.
- MediScan is an aid, not a medical device. A Verified result still needs a pharmacist's or doctor's confirmation.

CHAPTER 6
MODEL DEPLOYMENT

6.1 DJANGO WEB APPLICATION

MediScan is deployed as a Django web application. The recognisers are loaded once when the server starts, so each photograph only pays for the reading, not for loading a two-gigabyte model. The application has one upload page and one result page. Table 6.1 lists the main files.

6.1.1 UPLOAD PAGE

The user chooses the type of photograph (printed pack or handwritten prescription), selects an image file or takes a photograph with the phone, and presses the scan button. The server fixes the photo orientation and sends the image to the matching recogniser.

6.1.2 RESULT PAGE

The result page shows one of the four levels (Verified, Likely, Not found or Setup error) and, for each medicine found, its salt, strength, manufacturer, price, side effects and interactions. For a prescription, each line also shows the form, dose, frequency and duration, and the text that was read but not accepted as a medicine is listed under “other text read”. A note on the page tells the user to confirm every medicine with a pharmacist or doctor.

6.1.3 STANDALONE OPERATION

After the models and databases are copied to saved_models, the application needs no network. The Hugging Face libraries run in offline mode and EasyOCR loads its detection weights from disk. The photographs are held only in memory or in the local media folder. This is the main privacy benefit of the design.

Table 6.1: Main files of the application

File	Purpose
scanner/ml/recognisers.py	PrintedRecogniser and HandwrittenRecogniser; choose and call the right model
scanner/ml/handwritten_trocr.py	Line grouping, parse_line and prescription reading for Model B
scanner/ml/*init*.py	match_word: brand-word matching with the rules of Section 3.5
scanner/views.py	Upload handling, calls the recogniser and builds the result
templates/scanner/result.html	Result page with the four result levels
saved_models/	modelA_printed_ocr, modelB_handwritten_ocr, medicines.sqlite, drugs.sqlite

6.2 GIT HISTORY

The source code of MediScan is maintained with Git for version control. Repository:

https://github.com/KarthikUnnikrishnan/MediScan

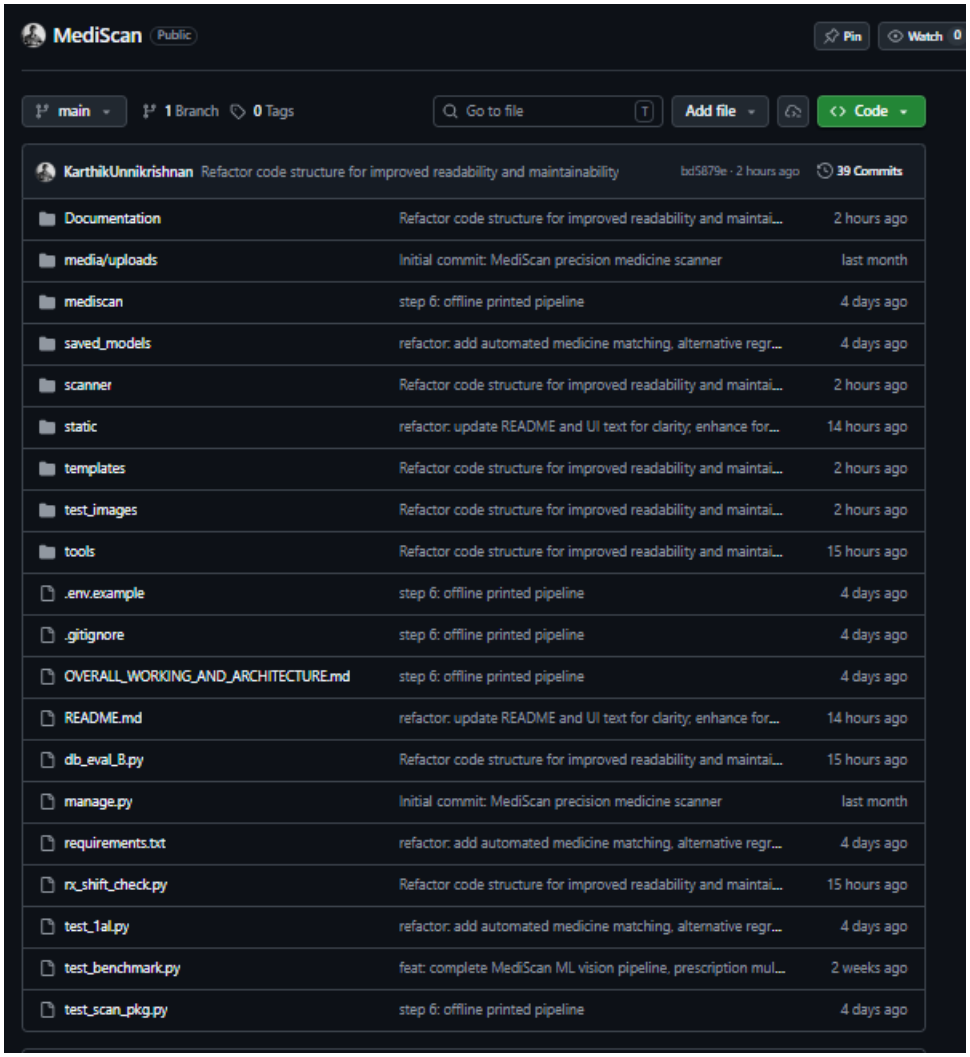


Fig 6.1: Git commit history of the MediScan repository

CHAPTER 7
CONCLUSION

MediScan shows that a complete medicine reader can run on an ordinary computer without any internet connection. Two fine-tuned TrOCR models, one for printed packs and one for doctors' handwriting, are joined by a text detector, a brand-word index and a local database of 503,536 medicines.

The project began from a simple need: a patient, a caregiver or a pharmacy assistant should be able to photograph a medicine strip or a prescription and quickly learn what the medicine is, without typing a name and without sending personal health information to an outside server. All the objectives set in Chapter 1 have been met. A clean training set was built for printed labels from the agreement between two readers and from synthetic images made from the medicine database. A large handwriting set of 177,057 images was collected, with a test set that holds only real prescription words. The training-loss problem in the TrOCR library was found and fixed, a matching layer was added to turn a noisy reading into a database medicine, and the results are shown in four honest levels.

The main results are as follows. Model A reads real label crops with 84.9% exact match and 1.83% character error. On 108 pack photographs the final matcher raises the precision of a "Verified" answer from 66.7% to 83.3%. Model B, trained on 177,057 images, reads the real-only test set with 14.8% character error and 73.0% exact match, and 82.8% of in-database words are the first suggestion after matching. The average time for a prescription photograph is 5.1 seconds.

Fine-tuning proved worthwhile for the final answer and not only for the text reading. On the comparison photo set the fine-tuned Model A found the correct medicine as the first suggestion for 50.0% of the photos, against 41.3% for the pre-trained printed reader. The matching rules were tuned on one half of the 108 benchmark photographs and checked on the other, untouched half, so the improvement is not a result of tuning on the test data. The most useful change was in the meaning of the word "Verified". With plain matching every photograph was called verified, but with the final matcher the system says "Likely" or "Not found" when it is unsure, instead of giving a confident wrong answer.

The handwriting results show both the strength and the limit of the approach. Printed-style prescription words are read very well, with 95.7% exact match, while the harder RxHandBD

doctors' handwriting is read with 57.8% exact match. Words seen in training are read correctly 82.8% of the time, but words never seen only 38.9% of the time. This is why the database matters: the medicine database acts as the vocabulary of the application, and the matching layer recovers many near-miss readings. The precision gate and the stop list, added after early runs showed English words and header text appearing as medicines, keep such text out of the medicine list while still showing it under "other text read".

On the engineering side, the application was made fast and fully local. By limiting the number of text boxes, reading in batches of 16, using fp16 and two beams, the average time for a prescription photograph fell from 14.9 seconds to 5.1 seconds. The models are loaded once when the Django server starts, and after the model and database files are copied to the saved_models folder the system needs no network. All the software is open source, no paid service is required, and training was completed on free GPU time, so the system has no recurring cost for the user.

An important lesson of the project is that careful engineering around the model matters as much as the model. The loss correction, the leak-free test sets, the label-shift check, the precision gate and the four result levels made the system more trustworthy than raw accuracy figures alone would suggest. The project also shows what is still missing: a measured whole-prescription precision and recall, and a larger set of real doctors' handwriting.

MediScan is an aid and not a medical device. A Verified answer still needs to be confirmed by a pharmacist or a doctor, and the application says so on every result page. Within this limit, the project has shown that a private, offline and honest medicine reader is practical on ordinary hardware, and it provides a base on which the improvements described in the next chapter can be built.

CHAPTER 8
FUTURE SCOPE AND ENHANCEMENTS

The most important direction for future work is more real handwriting. Words that were never seen in training are read correctly less than 40% of the time, and the doctors' handwriting in the RxHandBD set is still read much less accurately than printed-style words. Collecting and labelling more real prescriptions, with the consent of the patients and doctors, would give the model a wider range of writing styles, short forms and unusual spellings. A larger and cleaner set of real test words would also give more reliable figures, and the few shifted labels found among the RxHandBD test images can be corrected at the same time.

At present the accuracy is measured for single words and for medicine lookups. The next step is to complete the ground-truth file and to report the precision and recall of the listed medicines on a larger set of whole prescriptions, so that the performance of the complete application can be stated with confidence. Along with this, the interactions database can be used to check every pair of medicines read from one prescription and to warn the user about combinations that should not be taken together. This feature should be tested on real cases and reviewed by a pharmacist before it is relied upon.

MediScan now runs on an ordinary laptop, and each recogniser is a large model of about two gigabytes. Compressing the models through distillation and quantisation would reduce their size and the time taken for each photograph, so that the complete application can run on a phone or on a low-power computer. A mobile version would let a person scan a strip at the pharmacy counter or at home, and it would keep the main advantage of the present design, which is that no photograph leaves the device.

Many packs and prescriptions carry text in regional languages as well as in English. Adding recognition of such text, together with the matching names in the medicine database, would make the system useful to many more people. Some tablets also carry no printed text at all. A visual identifier based on the shape, colour and marking of loose tablets could be added for these cases, so that the system can help even when there is no label or strip to read.

The medicine database and the matching layer can also be improved. New brands, changed prices and newly reported interactions appear all the time, so a simple method of updating the local database would keep the answers current while the rest of the system still works offline. The brand-

word index could be extended with common misspellings and short forms collected from real prescriptions, and the rules for the four result levels could be adjusted as more whole-prescription results become available. Each such change should be checked on the leak-free test sets before it is accepted, as was done in this project.

The interface can be made easier for people who need it most. Large text, spoken output of the medicine name and its use, and simple warnings in the local language would help elderly users and those who cannot read small print. The result page could also show all the medicines of one prescription together with their timings, so that a caregiver can follow the doses easily. In every such change the present principle should be kept: the system must say honestly how sure it is, and the user must always be advised to confirm the medicine with a pharmacist or a doctor.

A pharmacist review mode is another useful addition. A pharmacist would be able to correct a wrong result and confirm a right one, and these corrections could then be used to improve the models and the brand-word index. This would create a steady cycle in which the system learns from real use, and it would also give a better measure of how often each result level is correct. With these improvements MediScan can grow from a working project into a dependable aid for patients, caregivers and pharmacists.

CHAPTER 10
REFERENCES

[1] R. Smith, “An overview of the Tesseract OCR engine,” in Proc. 9th Int. Conf. Document Anal. Recognit. (ICDAR), 2007, pp. 629–633, doi: 10.1109/ICDAR.2007.4376991.

[2] B. Shi, X. Bai, and C. Yao, “An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition,” IEEE Trans. Pattern Anal. Mach. Intell., vol. 39, no. 11, pp. 2298–2304, 2017.

[3] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks,” in Proc. 23rd Int. Conf. Mach. Learn. (ICML), 2006, pp. 369–376.

[4] A. Dosovitskiy et al., “An image is worth 16×16 words: Transformers for image recognition at scale,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2021.

[5] A. Vaswani et al., “Attention is all you need,” in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, 2017.

[6] M. Li et al., “TrOCR: Transformer-based optical character recognition with pre-trained models,” in Proc. AAAI Conf. Artif. Intell., vol. 37, no. 11, 2023, pp. 13094–13102, doi: 10.1609/aaai.v37i11.26538.

[7] Y. Baek, B. Lee, D. Han, S. Yun, and H. Lee, “Character region awareness for text detection,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2019, pp. 9365–9374.

[8] U.-V. Marti and H. Bunke, “The IAM-database: An English sentence database for offline handwriting recognition,” Int. J. Document Anal. Recognit., vol. 5, no. 1, pp. 39–46, 2002.

[9] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.

[10] M. Bachmann, “RapidFuzz: Rapid fuzzy string matching in Python and C++,” software. [Online]. Available: https://github.com/rapidfuzz/RapidFuzz

[11] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2019.

[12] P. Micikevicius et al., “Mixed precision training,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2018.

[13] T. Wolf et al., “Transformers: State-of-the-art natural language processing,” in Proc. Conf. Empirical Methods Natural Lang. Process.: Syst. Demonstrations, 2020, pp. 38–45.

APPENDIX

SCREENSHOTS

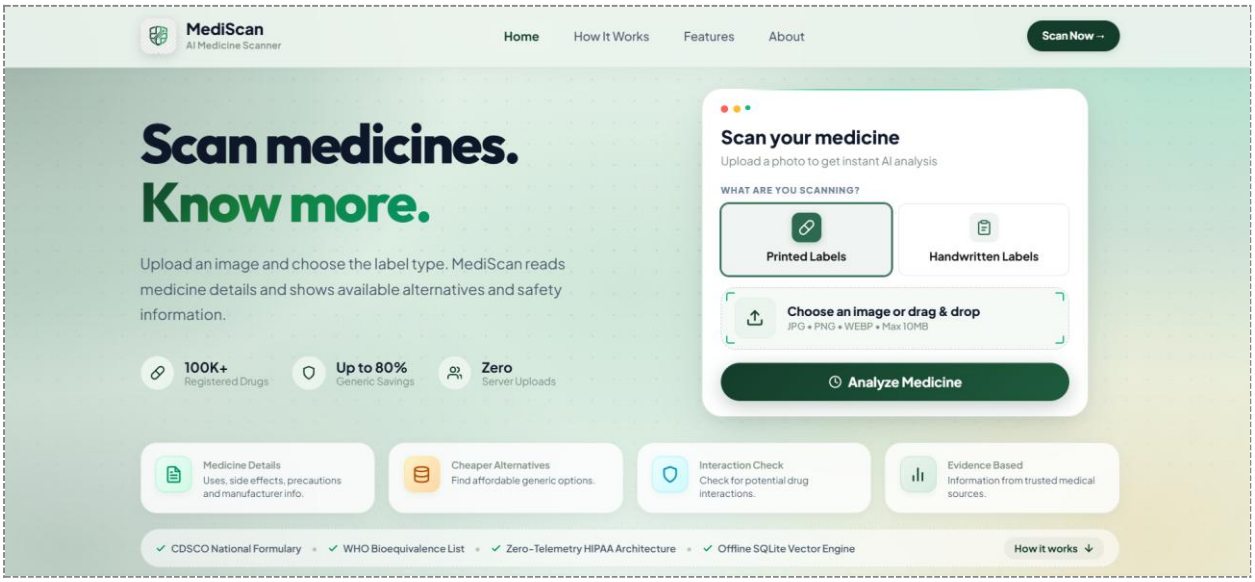


Fig 10.1: Home page

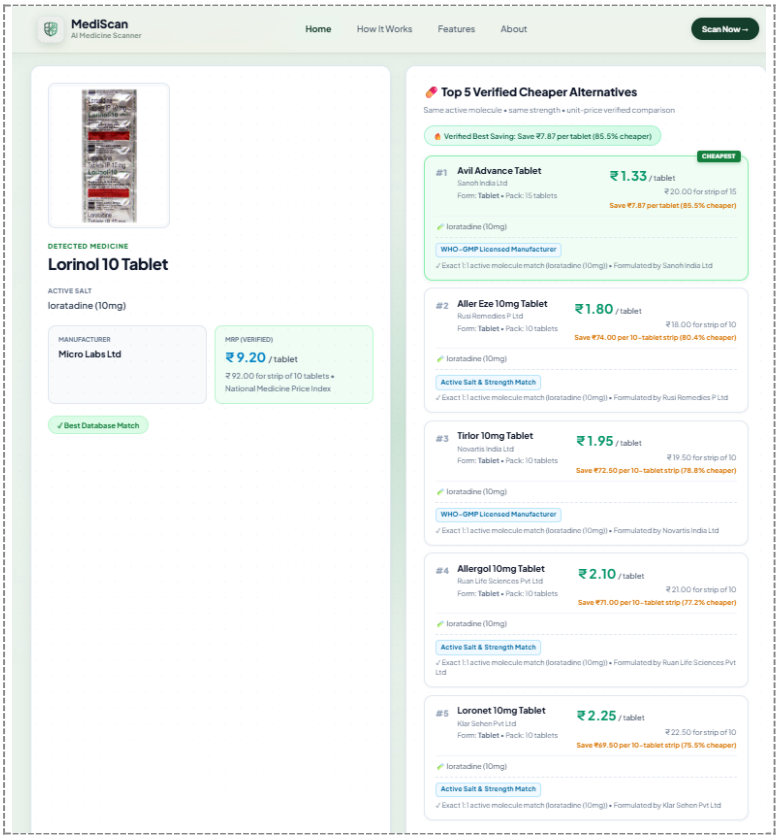


Fig 10.2: Printed pack result

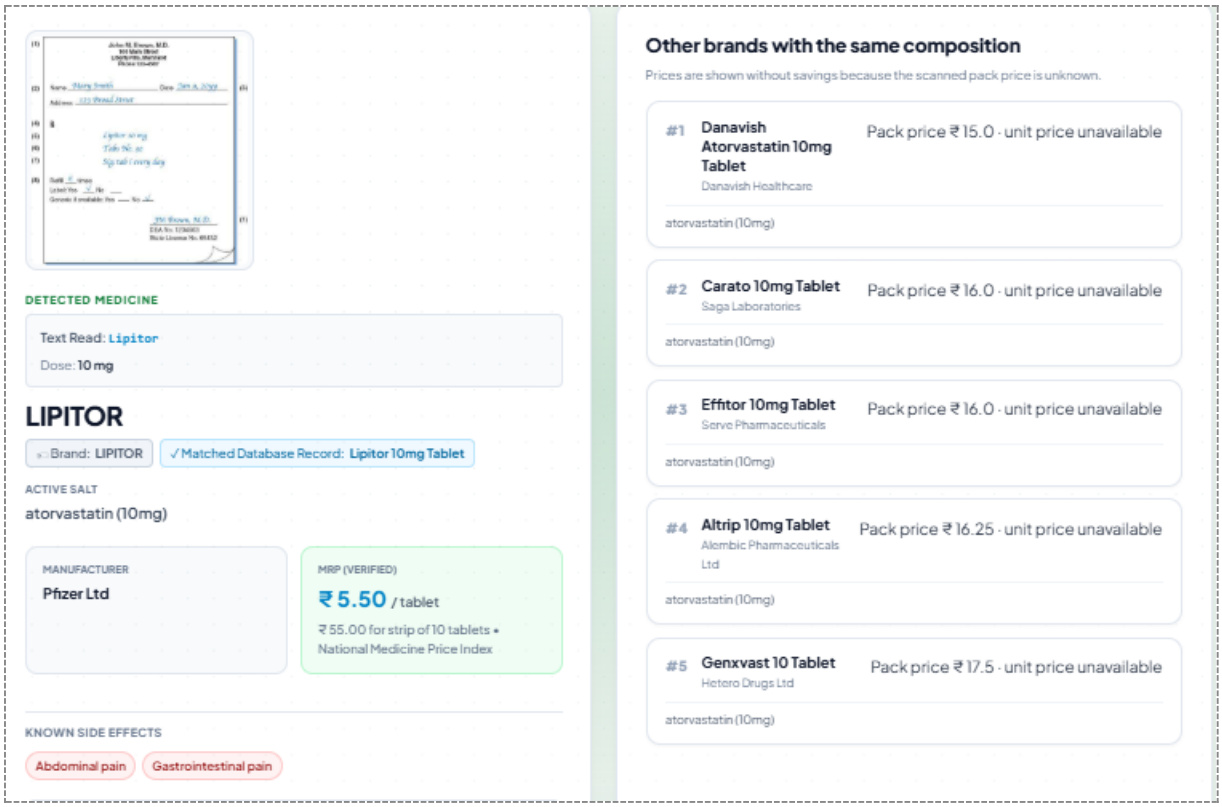


Fig 10.3: Handwritten prescription result

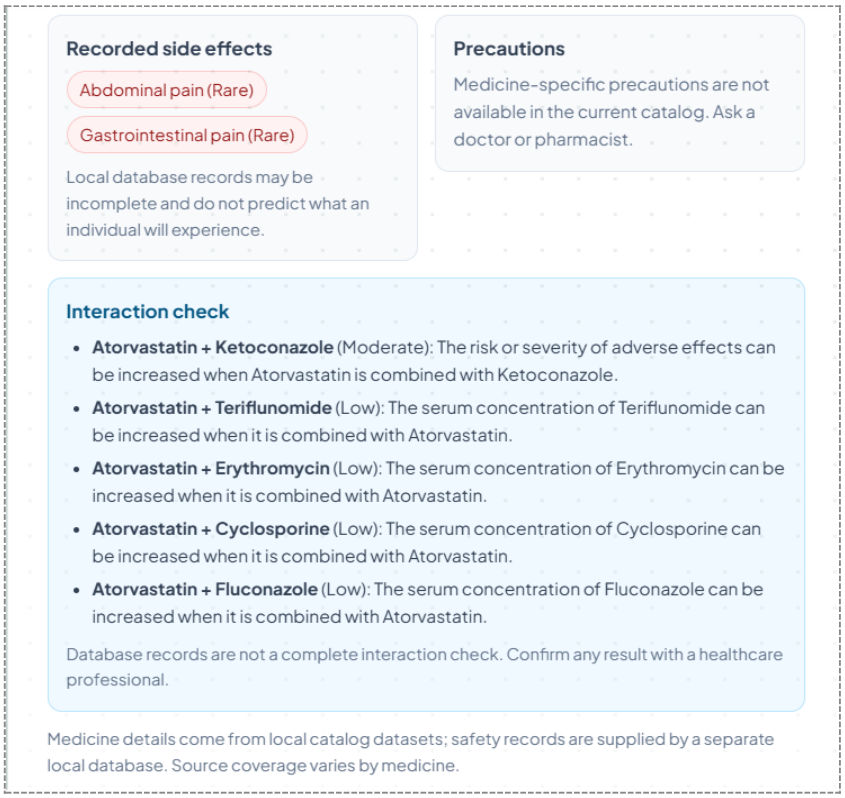


Fig 10.4: Medicine details and interactions